# Big Data Management
# Building a Big Data Architecture

## Project guidelines (P2): trusted zone, exploitation zone and downstream tasks

The structure for the P2 document will be slightly different. First, we will introduce, conceptually, the remaining stages of our architecture (section 1). Then, we will describe how these layers can be implemented (section 2). This includes the overall design, storage selection and the transformations between zones. As a complementary part of the pipeline, we present data governance tasks (section 3), which ensure an effective and efficient use of data. Finally, we detail the deliverables (section 4).

# 1. Stages (P2)

## 1.1 Trusted Zone

Raw data comes with all sorts of mistakes or inconsistencies that need to be addressed before it can be used in downstream tasks. This *pre*processing is an essential step of any pipeline that handles data in some capacity, and you have probably seen many techniques to implement this step in other courses. Nonetheless, when handling big data we have to take into consideration that some of our data is likely going to be used by different exploitation tasks. Hence, executing transformations that prepare data for a **specific** application (e.g. training a given ML model) is not a recommended practice, and should be left for further down the pipeline.

A useful operational heuristic: transformations in the Trusted Zone should be **information-preserving and reversible in spirit**. Fixing a malformed date, casting a string to integer, or removing a duplicate row do not destroy analytical signals; they correct representational issues. In contrast, removing outliers, imputing missing values, normalizing image brightness or aggressive text normalization are lossy transformations that encode assumptions about the downstream task, and therefore belong in the task-specific processing layer (i.e. closer to the consumption stage). When in doubt, ask: if two downstream tasks with different requirements consumed this data, would both benefit from this transformation, or would one of them be harmed?

As a result, in the trusted zone the goal is to conduct ***generic* data quality tasks** that do not interfere with posterior analytical needs. Another way to think about this is as "correcting the mistakes" that the data contains as a result of its unprocessed ingestion in the landing.

### Cleaning the data

Note that the specific processing tasks to be conducted depend on the data that is being handled. For example, structured data such as CSV files can undergo a lot of transformations to ensure its quality, given the constraints associated with the tabular format and a fixed schema. On the other hand, semi-structured data, such as JSON files, might be trickier to process given their variable schema. We now list some examples of tasks that can be conducted in this zone:

Structured data:

- Deduplication: removing repeated elements (e.g. based on primary key)
- Schema correction: ensuring data is casted to its proper type.

- Standardization (e.g. define a common format for dates, perform currency conversion or translate text to a specific language).
- Schema flattening (e.g. explode arrays, extract nested fields).

Semi-structured data:
- Schema alignment (e.g., "user_id" vs "userid" vs "id").
- Convert deeply nested objects into a more accessible tabular format when appropriate.
- Handling missing or optional fields (e.g. add default values).
- Ensure the data conforms to expected structural rules (e.g. valid JSON schema).

Unstructured data:
- Check file integrity and remove / fix corrupted files
- Text: convert to lowercase, remove extra whitespace, standardize encoding, etc.
- Images: resolution and size normalization, color / brightness correction, etc.

**Constraint 1: Trusted Zone design.** First, select the required data storage tools (see section 2) to implement the Trusted Zone. Second, define where each of your data assets will be stored and which transformations will be applied to each to clean it.

**Constraint 2: Trusted Zone implementation.** Set up the required tools and implement the pipelines that move the data from the Landing Zone to the Trusted Zone. Do so with the appropriate technology (see section 2).

Factors that will **positively contribute** towards the grade:
- Thoroughness and correctness of the cleaning tasks.

## 1.2 Exploitation Zone

So far our data has been ingested and cleaned. Our goal in the Exploitation Zone is to *curate* the data, that is, to provide the data in the best way possible to conduct analysis. This is important because until now we have kept the data mostly as it was ingested (besides some high-level structuring and cleaning), which means that it is not yet optimized to be handled by analysts.

This zone is going to involve two main tasks. First, data is going to be organized, modeled and stored in the appropriate way to facilitate its later use. Secondly, we might compute new data or structures that provide additional information to the analyst.

### Organizing

The organization of the Exploitation Zone highly depends on the data that you have, and there is no clear pattern to do so. This lack of a one-size-fits-all approach emphasizes the importance of adapting the structure based on the specific nature of your data. It is useful to think about it as the **ground truth** for subsequent tasks, that is, the data that will be given to the analysts for them to perform whatever processes they need.

The first differentiation is regarding the type of data used. Unstructured data has no schema, so all we can do to facilitate the analyst's work when selecting the data to work with is to store it in buckets that clearly indicate the data that is contained. The same structure employed in the Landing Zone might suffice, or some, more specific, buckets might need to be created.

Structured and semi-structured data, on the other hand, have a schema, so the organization can be both at the file level as well as at the schema level (modeling). Let's see some examples. A common operation is to integrate data from multiple sources that describe the same entities (e.g. joining patient visit records with diagnostic results on patient identifier) to produce wide, query-friendly tables that minimize the need for joins at consumption time. This **denormalization** is typical of the Exploitation Zone: unlike operational databases, where normalization reduces update anomalies, analytical workloads benefit from pre-joined structures that reduce query complexity and latency. Star or snowflake schemas are classical instantiations of this idea. In specific cases, **vertical partitioning** may still be useful, for example, isolating large or rarely accessed attributes (blobs, long text) into separate tables to keep the main analytical table compact. This is the exception rather than the rule.

Similarly, different versions of the same dataset belonging to different timeframes (e.g. country economic data; one dataset for each month, each with the same set of attributes) can be combined (i.e. **table union**) into a single table that contains all the historical records of that dataset. On the other hand, a table can be **partitioned horizontally** (e.g. based on a criterion such as geographic region) to provide smaller datasets tailored for more specific analyses, which can help optimize storage and performance by grouping similar records together.

Structured data's consistent schema makes these transformations and integrations relatively straightforward. With semi-structured data, such as JSON or XML files, the process is somewhat more complex due to the flexible and potentially nested structure of the data. However, since there is still an implicit schema present, some level of transformation and organization is possible. For instance, fields that appear consistently across entries can be extracted and normalized into structured tables, enabling similar operations like joins or partitions to be performed once the semi-structured data has been partially structured.

## Computing new data

Besides organizing the data, we can use the Exploitation Zone to calculate additional assets that provide extra benefits for the analysts. The idea is to generate information that can potentially be used by a wide variety of tasks, preventing the need to re-compute these values every time the analytical process needs to be conducted, thereby improving efficiency and consistency.

When working with unstructured data we can enhance its analytical value by extracting structured representations that semantically describe its content. For example, textual data can be summarized or classified into categories (e.g., "review", "complaint", "inquiry"). Similarly, image data can be categorized or labelled in some way (e.g., "cat", "dog", "vehicle"). In both cases, data can be stored as **embeddings** that capture semantic

meaning. By storing these semantic characterizations in the Exploitation Zone, we prevent having to repeat these computationally expensive operations for each analysis.

For semi-structured and structured data, the exploitation zone can also serve as a staging ground for calculating higher-level metrics and aggregates. Leveraging the existing schema, we can compute and store **key performance indicators (KPIs)** and other summary statistics that are relevant to the organization's strategic goals. These might include metrics such as average sales per quarter, churn rates or user engagement trends. By maintaining a historical record of these aggregates, analysts gain immediate access to essential insights that reflect the organization's performance over time, without needing to recompute them with each analysis.

**Constraint 3: Exploitation Zone design.** First, select the required data storage tools (see section 2) to implement the Exploitation Zone. Second, define how the data is going to be organized, both in terms of its storage (e.g. databases, buckets) as well as its modelling (e.g. should we perform joins? Should we split some data?). Third, decide which additional data assets you are going to generate (e.g. KPIs, embedding, labels, etc.) and how.

**Constraint 4: Exploitation Zone implementation.** Set up the required tools and implement the pipelines that move the data from the Trusted Zone to the Exploitation Zone. Do so with the appropriate technology (see section 2).

Factors that will **positively contribute** towards the grade:
- Effort in providing a coherent and nuanced organization to the data.
- Appropriate development of new data/artifacts

**Scope of the Exploitation Zone.:** The amount of restructuring and new assets you can introduce depends on your data and use case; we do not expect extensive transformations or large catalogs of derived artifacts. What we evaluate is whether the organization you propose is coherent with the consumption tasks you have defined and whether the design decisions are well justified.

## 1.3 Data Consumption

At this point most of the data management aspects have been fulfilled, and data is ready to be employed in subsequent tasks or fed to external systems. As mentioned in previous sections, **the focus on this course lies in the management aspects rather than on the analytical processes** and, hence, the quality or thoroughness of the steps implemented here are not the main focus on the project.

The priorities for this stage are that you think about how to exploit your data (you should have a clear idea of this at the beginning of the project), that the systems to do so are set up (i.e. you have a place to move the data to), and that the adequate connections between the Exploitation Zone and the data consumption mechanisms are in place (i.e. something gets in and, ideally, something gets out).

For example, if you decide to train an ML mode, we will check whether the data is properly shipped to the model, some training process is conducted and the resulting model is stored somewhere. The thoroughness of the training process or the quality of the final model will not be analyzed. If time becomes a limitation, it is also acceptable to have "placeholders" rather than exploitation systems. That is, set up a system in which some data gets in and, without any processing, something gets out, mimicking the result of the process that should be conducted.

Some examples of tasks you can implement in this step are:

- Training a machine learning model.
- Building a dashboard.
- Setting up an alert system (real-time monitoring system that triggers alerts).
- Developing a recommendation system.
- Powering a search engine or query system (make the data searchable).
- Natural language processing (NLP) Tasks.
- Building a chatbot with RAG that accesses your data
- Building an image recognition system.

**Constraint 5: Data consumption design and implementation** First, define which mechanisms you are going to use to exploit the data. Then, select the tools you will use to implement such mechanisms and set them up. Finally, ship the data from the Exploitation Zone onto selected systems.

Factors that will **positively contribute** towards the grade:
- Setting up minimal, although functional, tasks rather than placeholders.

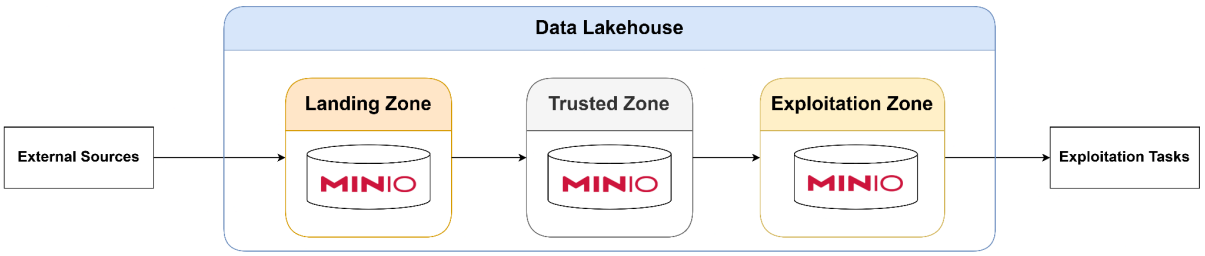
# 2. Implementing the zones

## 2.1 Design

The most important part of the implementation is that each zone **has to store the data after the respective transformations have been applied**. That is, you have to define separate storages for each zone. For example, in the Trusted Zone you get the data from the Landing Zone, apply the necessary cleaning tasks, and store the curated data in a separate place. Similarly for the Exploitation Zone. In this way we create checkpoints that we can go back to access the data in different stages of maturity. In this project we present two main ways of proceeding with the implementation.

### 1st option: extending your current architecture

The Landing Zone implemented in P1 is mimicking the structure of a realistic data management pipeline: storing data in an object store, with metadata handling on top, orchestration of the execution, etc. (the only exception being that we are not using cloud environments). **This existing foundation can be extended** to implement the remaining zones in your pipeline. Take the following simplified diagram as an example:

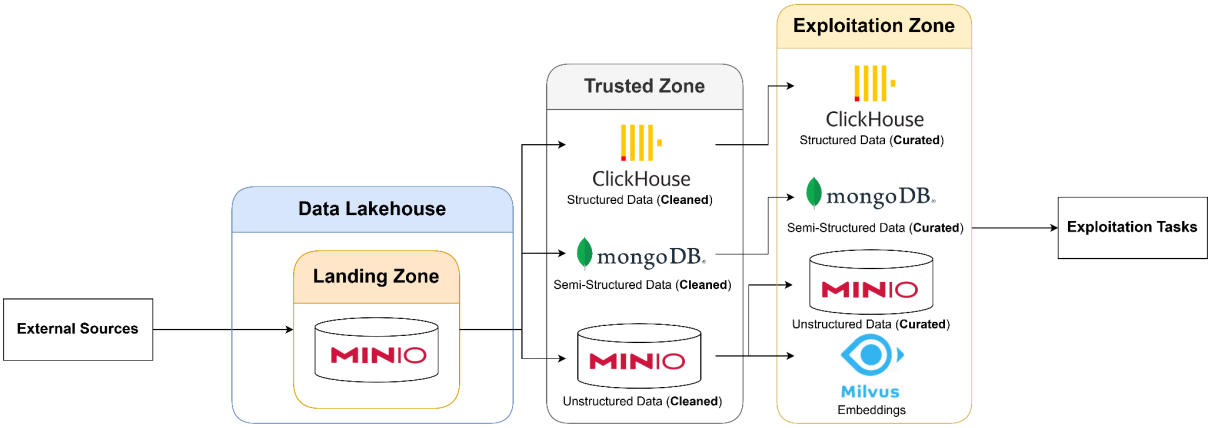



The idea is to take your current system and include more folders / buckets to represent the next zones. The advantage here is that you continue working within an ecosystem you are already familiar with, reducing the learning curve and simplifying integration. Moreover, this design is the **most common in *real* data management systems**, where the flexibility of object stores prevents the need of including more specific technologies. However, as we are in an academic environment, we present a second approach that will allow you to further play with the design of the system and utilize more tools.

## 2nd option: moving to smaller, more specialized databases

In traditional data management, before even distributed processing, the pipelines were centered around **data warehouses**. However, these stores could end up containing a lot of data, which could slow the query response time and complicate data governance. Moreover, the data ingested by the system might belong to widely different domains, so having it all mixed up in a single environment was not ideal. As a response to that, designers came up with the concept of ***data marts***, subsets of the data warehouse that focuses on a single business function or domain. These smaller repositories are streamlined for quicker querying and a more straightforward setup, catering to the specialized needs of a particular team or function.

We can employ this idea of smaller, separated and more specific databases to design a different structure, one which implements the Trusted and Exploitation Zones in **specialized systems** outside of your chosen technology for the Landing Zone. This division can be, for example, based on the format of the data:




In the above image we use ClickHouse (a SQL database), MongoDB (a document store) and Milvus (a vector database) to handle each specific data type, providing

functionalities that an object store can not. In the next section we provide a list of several database types and recommended examples to implement the remaining zones.

Following this idea, you can design the structure that you want based on the criteria that you deem fit. This model will allow you to make more complicated structures, which implies more work on the design aspects, and force you to use different technologies, which is great if there are tools that you want to explore to expand your skill set. Going back to the previous diagram, note that, for example, we are storing embeddings for the unstructured data in a specific database (Milvus), separated from the original data (which is still in MinIO). We could have stored the embeddings in MinIO as well, but by introducing a vector database we provide a much easier way to query and retrieve the embeddings. A similar thing is happening for the structured and semi-structured data, as by using dedicated systems we provide a better handling for these assets. The tradeoff is that introducing these technologies complicates the architecture.

**Constraint 6: Architecture design.** Continuing with the work you did with part 1, you have to complete the architecture diagram with the new zones, tools, operations, etc. Be sure to adequately reason about the general structure you propose and justify the inclusion of each element.

Regarding the presented design alternatives, option 1 is more straightforward, whereas option 2 can allow you to make a more interesting pipeline. From an evaluation standpoint, both are perfectly acceptable. Also note that you can find an **example of the full project diagram** at the end of this document.

## 2.2 Databases

The choice of database depends on the specific type of data and the use case at hand. Traditionally, relational databases dominated the landscape, but with the growing complexity and volume of data, newer database models (i.e. NoSQL) have emerged to better handle diverse requirements. Next, you can find a brief overview of the main typologies and some recommendations. Before implementing the pipeline you should decide what type of databases better suit your needs.

**Relational databases**: you should be familiar with this model at this point. Tools like Oracle, MySQL and PostgreSQL still lead the charts for the most popular systems to store data. Nonetheless, these databases store data row-wise, which makes them great for dealing with transactions (OLTP), but not so much to deal with analytical workloads (OLAP, mainly based on aggregations). To fix this, “modern” relational databases have been created, which have columnar storage, seamless integration with other data science tools and optimized for analytical queries. Our recommendation is that you try some of these new tools. Two good options are **DuckDB**, an embedded, file-based database that is very easy to use, and **ClickHouse**, a server-based columnar database designed for larger analytical workloads.

**Key-Value stores**: a non-relational database where records are stored and retrieved based on a unique key, similar to a hash map or dictionary. This structure allows for quick

lookups and is well-suited for caching and other fast-access requirements. The most popular key-value store is **Redis**.

**Document stores**: a specialized key-value store, where a document is a nested object (we can think of each document as a JSON). Documents are stored in collections and retrieved by key. Document stores are great for semi-structured data like JSON or XML, and allow for flexible schema design. **MongoDB** (which you have seen during the course) is the most popular document store, and ideal if you need fast read / write queries.

**Wide-column**: optimized for storing large amounts of data with high write throughput and low latency, wide-column stores are designed for scalability. The columns and schema of tables allow for far more flexibility and runtime freedom (if compared to relational databases). These stores are designed to efficiently store vast amounts of unstructured or semi-structured data. During the course you have seen **HBase**, which can be set up outside of the Hadoop environment and in an embedded mode. Another alternative is **Apache Cassandra**, which is the most popular wide-column store.

**Graph databases**: explicitly store data with a mathematical graph structure (as a set of nodes and edges). Roughly speaking, graph databases are a good fit when you want to analyze the connectivity between elements. **Neo4J** is a good option, as you have seen / will see it in other courses and it is the most popular graph database.

**Time series**: a time series is a series of values organized by time. For example, stock prices might move as trades are executed throughout the day, or a weather sensor will take atmospheric temperatures every minute. Any events that are recorded over time are time-series data. A time-series database is optimized for retrieving and statistical processing of time-series data. The most popular tool is **InfluxDB**.

**Vector databases**: stores and queries embeddings or vector representations of data. These vectors are typically generated using machine learning models like BERT that map data into high-dimensional space. Vector databases are becoming increasingly popular, especially in fields like NLP and search systems that require high-performance similarity search for multi-dimensional vector data. A good option is **Milvus**.

## 2.3 Transformations

Once we have defined how the zones will be implemented, it's time to talk about how to generate the transformations that data will undergo while travelling between zones.

### Spark

When working with centralized data, libraries like Pandas or NumPy are perfectly suitable. However, when datasets become too large to fit on a single machine and must be distributed, these libraries are no longer sufficient, and we need a new paradigm that distributes computation across multiple nodes.

During the course, you will be introduced to **Apache Spark**, a leading technology in the big data ecosystem that is widely used for **distributed data processing**. Spark was inspired by the MapReduce paradigm but extends it with a more flexible approach. It is

designed to handle large datasets across clusters of machines, making it well-suited for both batch processing and near-real-time (micro-batch) pipelines.

You may notice that defining data transformations in Spark differs significantly from working with familiar Python libraries like Pandas. While Pandas can offer simplicity because it deals with in-memory data processing, having to deal with distributed data and its characteristics makes the Spark code feel more verbose and complex, but this is intentional and essential for ensuring that data workflows remain efficient and reliable. In the theoretical part of the course, you will learn how Spark achieves these properties.

### Streaming

*Conceptually*, if you define a hot path (i.e. real-time), data should be sent without delay, so any processing that we did, even if minimal, would not be desired, as the data shipping would be slowed down. On the other hand, a warm path, in which the latency is higher, can more comfortably accommodate some processing before moving the data to its final destination. In a *practical* setting, whether to process the data or not depends on a lot of factors, and there is not a clear, general answer.

For your project, you can decide to process or not the data you stream. You can justify your decision based on the above criteria or any other you deem appropriate. If you decide to apply some processing, we recommend checking the following tools:

- **Spark**: the Spark core installation is equipped with functionalities to ingest streamed data (from Kafka, for example), process it and send it to another streaming pipeline. You will be using Spark for the batch-ingested data regardless, so it is a good option to also employ it here.
- **Kafka Streams**: a complimentary tool inside the Kafka ecosystem to natively process the Kafka streams. It is more lightweight and specialized than Spark, but it is implemented in Java. If you are not familiar with the programming language, it is best to avoid it.
- **Apache Flink**: a stream-processing engine with a native streaming execution model (as opposed to Spark's micro-batch approach), which typically yields lower latencies on true streaming workloads. It can also handle batch processing as a special case of streaming, and exposes a Python API (PyFlink).

Processing technologies like Spark or Flink can consume data from Kafka streams, perform their transformations and can either write the results directly to a database or publish the processed data to another Kafka topic. This allows you to continue moving data downstream in a fully streaming manner.

# 3. Governance (Optional)

Some aspects of the project, such as tools like OTFs and tasks like metadata handling, are related to **data governance**. In this section we will present a more in-depth exploration of what data governance entails, given that it plays a pivotal role in large-scale data management. Note, however, that the governance tasks we describe below **are not mandatory**, and including governance processes will simply **positively contribute** towards the grade. Please check the guidelines at the end of the section.

As data pipelines expand to include more zones, tasks and technologies, they become more complex and harder to manage. To handle this complexity, organizations need **data governance**. Data governance is a **high-level structured framework that formalizes the management of data** through policies, standards, processes and ongoing monitoring. That is, we want to have control over the data that we have and process.

Data governance encompasses several key tasks such as data quality, security, access control, etc. In addition, governance must consider the type of data being managed (e.g. social media data, streaming data or sensitive biometric data), since different data types require different controls. When designing data governance, it is important to understand how these scopes tie together to maximize data value while minimizing risks and costs. This can quickly become complex and often involves navigating trade-offs between control, agility and costs.

A good starting point is to define your goals based on the business context. Some organizations adopt a centralized model, where a single team oversees how data is collected, stored, and shared. However, the right approach depends on priorities. For instance, if protecting sensitive customer data is critical, security will drive governance decisions. Nonetheless, if reliable analytics or machine learning is the focus, data quality becomes the priority.

Within this context, several key governance domains can be explored. In the following sections, we describe a set of data governance concepts and decision domains that are relevant at this stage of your architecture. For each, we also recommend tools and technologies to help you implement them.

## Data domains and data products

At this stage, your data should be clean and available in the Exploitation Zone. You may have already started organizing data according to business entities or anticipated downstream data science pipelines. Building on this, a common governance strategy in modern big data architectures is to **view data not simply as individual datasets, but as data products grouped within data domains**.

A data product contains the dataset itself and all the information related to it (e.g. ownership, documentation, objectives). It is designed to be consumed by other teams or systems, and is typically aligned with a specific domain or business process. Data products play an important role in governance, as rather than governing every table or pipeline independently, organizations can attach policies, ownership, quality checks and access control rules at the level of the data, providing a higher degree of control.

Organizing data products within the Exploitation Zone in data domains (e.g. sales domain) helps in adding another layer of organization since each data product is aligned with a specific area of business activity and thus, may have common standards, policies and metadata.

### Metadata

You have already seen the importance of metadata during P1. Metadata helps describe and organize the actual data and is a fundamental building block to ensure full control over your assets. Nonetheless, we presented metadata mostly as “extra information” for each dataset, but much more can be done with it.

Graph-based structures (which you have explored in SDM) can be used to describe metadata elements and their relationships, and several vocabularies and ontologies are available to standardize these descriptions. A good starting point is to design a data catalog that provides an overview of the available data products in the Exploitation Zone and their relationships. The **DCAT** ontology can be used to model and standardize this catalog, and tools such as **Apache Atlas** can be used to support its implementation.

Another interesting usage of metadata is to implement **lineage tracking or traceability**, which tackles on how data moves and transforms across the different zones, enabling to trace back the origin of a dataset, the steps it has undergone, and what were the tasks and interactions. This can be especially useful to debug data issues or meet audit requirements.

### Data Quality

Data integration and enrichment tasks can become complex processes that often involve numerous operations across various data sources. As such, errors or inconsistencies in these processes can lead to unreliable or inaccurate data being served to the exploitation zone. To mitigate these risks, monitoring tools can be used to ensure data quality throughout the pipeline. Tools such as **Great Expectations** can be integrated to define data quality requirements, validate data, and trigger alerts or corrective actions when quality issues are detected.

### Data Security

An essential aspect of governance is managing who has access to what data. This includes defining roles, assigning permissions, and implementing policies that restrict access based on user roles, data sensitivity, or zone location. For example, access to raw data in the Landing Zone may be restricted to data engineers, while curated datasets in the Exploitation Zone may be more broadly available to analysts and data scientists. Policies should reflect business needs and compliance requirements, such as GDPR.

Although data security and privacy are complex topics with rich research backgrounds, for the purpose of this project, you can imagine your organization and simulate governance roles and access levels across zones. Tools such as **Apache Ranger** can help enforce and monitor access control.

## Implementing data governance

Over your resulting data architecture, **begin to design your data governance strategy**. As previously discussed, data governance requires a shift from purely technical considerations to also include organizational scopes aligned with business ideas. In this phase, we are particularly interested in how well you align your governance proposals with your business context.

1. Take an organizational and strategic point of view. Begin by reflecting on the **data domains** that are relevant to your business. Then, based on these domains, propose a set of **data products** that should exist in your exploitation zone. Are these data products similar or different from what you initially had in your Exploitation Zone? What insights led you to adjust or reaffirm your original design?
2. Provide at least **one detailed example of a data product** and the metadata you would associate with it to support governance (e.g. description, owner, quality metrics, policies, data lineage, etc.). You may use this template to support your specification.
3. Choose at least **one governance mechanism** (e.g. data quality validation, access control, lineage tracking, cataloging) **and implement it** over your architecture. What are the benefits and potential trade-offs or costs associated with this mechanism?

We will pay special attention to your reasoning, justification of choices and how clearly you link governance elements to your business case.

# 4. Deliverables

## Main deliverable (P2)

The tasks described in this document correspond to the **second deliverable** (max. **12 pages long**), which include:

- **Architecture design** (updated version with the new zones, tools, processes, etc.)
- Stage 3: trusted zone
- Stage 4: exploitation zone
- Stage 5: consumption tasks
- Governance tasks (if applicable)

To implement each stage follow the appropriate descriptions.

**Constraint 7: orchestration and containerization (continued from P1).** The orchestration and containerization set up in P1 must be extended to cover the new zones and processes introduced in P2. That is, the end-to-end pipeline (ingestion → Landing → Trusted → Exploitation → Consumption) should be orchestrated and, where services are deployed, containerized. The evaluation criteria established in P1 (Constraints 8 and 9) continue to apply. As in P1, additional aspects such as error handling and reporting, execution monitoring or code quality tooling will positively contribute towards the grade.

The document must contain the following sections:

1. **Instructions**. Updated with the new steps / requirements.
2. **Architecture design**. Provide the updated diagram of your pipeline, including the appropriate explanations to understand the new parts of the architecture. The goal, once again, is to provide an overview of what you have implemented that can help in understanding your work (you can take the diagram on the last page of this document as reference).

3. **Trusted Zone**. Describe the design of the Trusted Zone, that is, the organization of the storage systems, how the data is shipped and the selected cleaning tasks for each data type. Include any relevant explanations about the implementation.
4. **Exploitation Zone**. Describe the design of the Exploitation Zone, that is, the organization of both the storage systems and the data assets, how the data is shipped and the new assets that have been generated. Include any relevant explanations about the implementation.
5. **Data Consumption**. Describe the downstream tasks that ingest the data from the Exploitation Zone.
6. (Optional) **Data Governance**. Describe the data governance mechanisms introduced, following the guidelines specified at the end of Section 3.

**Constraint 8. Organization.** Clearly organize your code and storage repositories per zones / stages. Each zone should have a folder with its respective scripts. Clearly separate the scripts per task (if / when required).

## Follow-up deliverables

There will be two follow-up deliverables in this second part, each before their corresponding follow-up sessions. This means that you will have roughly two weeks to deliver them.

**First deliverable**:

- Architecture design (first version).

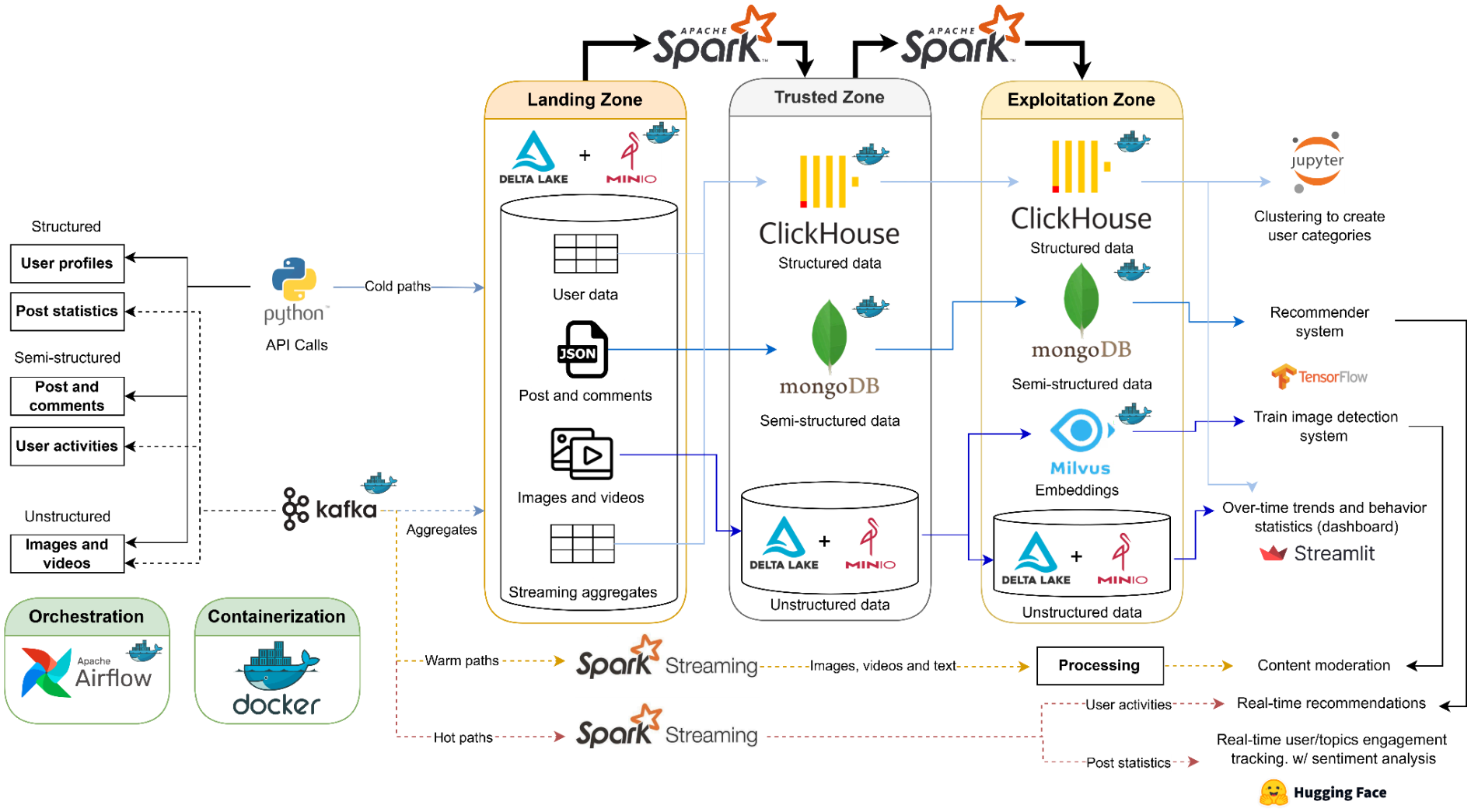
Again, the first deliverable does not need to include any code. Note that we highly encourage you to start the implementation as early as possible, as if you leave most of it for the last weeks it might conflict with other project deliveries and final exams.

Ideally, the design should include, besides the diagram, a clear view of what you want to do. That is, how are you going to set up each zone, which transformations are you going to apply to the data, how far are you going to go in each consumption task, etc. In summary, during the first two weeks you should go over the entire document and decide not only what you want to do, but how; with as much detail as possible.

**Second deliverable**:

- Architecture design (consolidated).
- Basic implementation of Trusted Zone, Exploitation Zone and Consumption tasks.

During these two weeks you should focus on setting up all the necessary infrastructure for the entire pipeline. That includes the databases, the downstream tasks, streaming pipelines and all the communications between the zones, with the necessary orchestration and containerization. The quality of the processing at each step is not the priority, but rather that you can move and store data where you need to.

The final two weeks should be devoted to implementing the actual transformations to ensure that the data undergoes all the desired changes, besides any additional aspects you might want to include (especially regarding data governance).

# 5. Pipeline design example